

Categories of categories

We now gather categories and functors into a category. We look at structures in that category, much as we have done with other examples of large categories of mathematical structures. We also show how this structure strains at its dimensions and is trying to expand into higher dimensions.

21.1 The category **Cat**

Categories are mathematical structures, and functors are structure-preserving maps between them. Good categorical instinct tells us that these “should” assemble into a category.

Things To Think About

T 21.1 What do we have to check to make entirely sure that categories and functors form a category? Can you do it?

We need to make sure we have an identity functor, and composition of functors, and that the axioms hold. For any category $\mathcal{C}$ we have an identity functor $1_{\mathcal{C}}: \mathcal{C} \rightarrow \mathcal{C}$ which acts as the identity function on objects and on morphisms. Composition also happens by composing the functions on objects and on morphisms. As this is all based on composition of functions it inherits the unit and associativity axioms from the functions.

However, there is one more technical issue with making a “category of categories”. Good mathematical instinct will make alarm bells ring here from the self-reference: trying to make a “set of all sets” lands us in a paradox (Russell’s paradox) and so there is a similar danger lurking if we try and make a “category of all categories”. We avoid this by restricting what size of category we think about at any given moment. We briefly mentioned this in Section 8.5.

The first level is *small categories*. We mentioned that a category is called *small* if its objects form a set (not a large collection) and its morphisms also form a set. We can then make a category **Cat** of small categories and all functors between them. However, this will not include any of the examples of “large categories of mathematical structures” like **Set**, **Pst**, **Top**. It also can’t include **Cat** itself, so we avoid a self-referential paradox.

The next level up is *locally small categories*. We said a category is called *locally small* if the morphisms between any two objects form a set; however the objects might form a (large) collection. For example, in the category **Set** the objects are all the possible sets, so these cannot form a set (to avoid Russell's paradox). However, given any sets A and B there is a set of functions $A \rightarrow B$. So **Set** is locally small. We can then make a category **CAT** of locally small categories and all functors between them.

Definition 21.1 We write **Cat** for the category of all small categories and functors between them. We write **CAT** for the category of locally small categories and functors between them.

We are going to look at some of the structure inside **Cat**, but first we will warm up by thinking about relationships between **Cat** and some other categories of structures we've seen. As in the previous chapter, it turns out we have been implicitly thinking about some of these functors already. This time the functors in question are between **Cat** and other categories.

Things To Think About

T 21.2 We have seen that posets are special cases of categories. Can you express this as a functor **Pst** $\rightarrow$ **Cat**? The analogous situation for monoids is more subtle; can you see why?

A poset is a category in which there is at most one morphism between any pair of objects. This means we have an “inclusion” functor **Pst** $\rightarrow$ **Cat** which simply sends each poset to itself as a category.

A monoid “is” a category with only one object, but now the situation is more subtle because there is a direct definition of monoid as a set with a binary operation satisfying some axioms. Unlike in the case of posets this is not *exactly* the same as a one-object category as we have performed a dimension shift and forgotten that the single object was ever there. This means that if we define monoids as sets with a binary operation, then to construct an “inclusion” functor **Mnd** $\rightarrow$ **Cat** we have to fabricate a single object from somewhere. We could decide that every monoid is going to become a one-object category with the same single object, say $*$. In any case this is why I put “is” in inverted commas at the start of this paragraph, because sending every monoid to itself expressed as a category is not quite as straightforward as for posets.

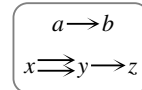
Things To Think About

T 21.3 What forgetful functors can you think of from **Cat** to **Set**? How sensible are they?

There are various forgetful functors from **Cat** to **Set** including functors that:

1. send a small category to its set of objects,
2. send a small category to its set of morphisms, or
3. send a small category to its set of *connected components*.

A connected component is a part of the category that is connected by arrows. For example the category on the right has two connected components (it's a diagram of just one category, although it might look like two).



You could try checking that each of the above three forgetful situations can be made into a functor, at least as far as seeing what happens on morphisms.

As for how sensible these functors are, that of course depends on what you mean by “sensible”. The forgetful functors **Grp** $\rightarrow$ **Set** and **Mnd** $\rightarrow$ **Set** were very sensible in the sense that groups and monoids really are sets with extra structure, so the relationship between the set with the extra structure and the set without it is what I am regarding as “sensible”. However, a category is not just a set with extra structure, because its underlying data has objects *and* morphisms. So the forgetful functors to **Set** are in a sense *excessively* forgetful. They don't just forget structure, they also forget data. A more “sensible” forgetful functor would only forget the structure of a category, not the data, and that means that we wouldn't land in **Set** but in the category of graphs.

This notion of “sensible” is not rigorous at all, but it's a feeling about the situation that we can have as humans, and developing those feelings is an important part of developing as a mathematician.

Recall that in Section 17.5 we expressed the underlying data for a category as a graph, that is, a diagram of sets and functions as shown on the right.

$$C_1 \begin{array}{c} \xrightarrow{s} \\ \xrightarrow{t} \end{array} C_0$$

Things To Think About

T 21.4 Can you make a category of graphs? What would be a sensible notion of morphism between graphs? If you're stuck, look back to what we said about the underlying data for a functor when categories are expressed in this way.

A morphism of graphs is given by a pair of functions making the diagram on the right serially commute; recall this means the square involving the s morphisms commutes and so does the one involving the t morphisms.

$$\begin{array}{ccc} C_1 & \begin{array}{c} \xrightarrow{s} \\ \xrightarrow{t} \end{array} & C_0 \\ F_1 \downarrow & & \downarrow F_0 \\ D_1 & \begin{array}{c} \xrightarrow{s} \\ \xrightarrow{t} \end{array} & D_0 \end{array}$$

Graphs and their morphisms then form a category **Gph**, and we have a sensible forgetful functor $\boxed{\text{Cat} \rightarrow \text{Gph}}$. What about a functor going the other way?

Things To Think About

T 21.5 Can you see how to make a functor going the other way, which makes a “free category” on a graph, just like we made a free monoid on a set? Which part of that construction do we need to modify to allow for the fact that we now have more than one object? Can you make the construction into a functor?

We are going to construct the “free category functor” $\boxed{\text{Gph} \xrightarrow{F} \text{Cat}}$.

It is a generalization of the free monoid functor $\boxed{\text{Set} \rightarrow \text{Mnd}}$.

For the free monoid functor we needed “answers” for the binary operation; for the free category functor we need to make “answers” for composition.

There is nothing to do on objects because a category does not have any extra structure on objects, only on morphisms. So if we start with a graph A , the free category FA has the same set of objects as A . For the morphisms we need to start with the morphisms of A and form composites freely (and an identity). This is just like making the free monoid in which we took strings of letters, except now we’re taking strings of morphisms and we only make strings of *composable* morphisms.

So if we just start with morphisms as shown on the right, then we don’t have to do anything because those morphisms are not composable. The free category on that data won’t need any extra morphisms for composites.

$$a \xrightarrow{f} b \quad x \xrightarrow{g} y$$

However, if we start with morphisms as shown here, then the free category on this data will need a composite $g \circ f$.

$$a \xrightarrow{f} b \quad b \xrightarrow{g} c$$

So in this case we need to add in a new morphism $a \xrightarrow{g \circ f} c$ to be the result of that composition. We will also add an identity at each object.

This is really only a sketch of the construction, but I hope the idea is clear.

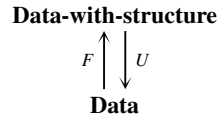
Definition 21.2 Given a graph A , the free category on it has

- objects: the same objects as A , and
- morphisms: finite composable strings of morphisms in A .

Note that finite composable strings include strings of length 0 (empty strings), which will be the identities. Composition is then given by concatenation, just like for words in the free monoid, but with the extra composability condition.

Aside on further theory

The relationship between free and forgetful functors is abstractly summed up as shown on the right and is very important.



At the top we have a category of “data equipped with a particular kind of structure”, together with structure-preserving maps. In good situations the free and forgetful functors are in a special relationship called an adjunction, which can be expressed via universal properties in various ways that are beyond our scope here. The composite UF is then a functor from the category of underlying data to itself, and has many excellent properties. It is a prototype example of what is called a *monad*. Monads are then an abstract way to study “sets with structure”, and more generally the category of data at the bottom could be something other than **Set**, in which case monads give us a very general way to study algebraic structure. In fact, at an abstract level this is a *definition* of algebra: something that can be studied via monads.

Now that we have assembled categories and functors into their own category we can look for our various types of structure in that category.

21.2 Terminal and initial categories

We are going to look for a few universal properties inside **Cat**, starting with our most basic universal property, terminal and initial objects.

Things To Think About

T 21.6 What do you think terminal and initial objects in **Cat** are? We have seen and even used them already.

For terminal and initial things in general, it is often productive to start thinking about “one-element” and “empty” and see how that goes. It doesn’t always work, as we saw with groups, but it can lead us to find what works.

For categories we need to think about objects and morphisms, so for the case of “one-element” it’s a good idea to think about one object and one morphism, that is, the category we have already met[†] and called **1**.

Given any category $\mathbb{C}$ there is a unique functor $\mathbb{C} \rightarrow \mathbf{1}$ because all objects

[†] We used it when we were finding functors that did not reflect isomorphisms and commutative diagrams.

of $\mathbb{C}$ must go to the single object of $\mathbb{1}$, and all morphisms must go to the identity. Functoriality holds because everything is the identity, so all the required equations become “ $1 = 1$ ”.

For the initial category we can try taking the empty category 0 , which has no objects and no morphisms. There is then a unique functor to any category $\mathbb{C}$ as it is vacuously defined, just like the “empty function” from the empty set to any set. Functoriality is then vacuously satisfied.

We have seen that the terminal and initial objects in **Cat** are as we “expected”: the category with one object and one morphism is terminal, and the empty category is initial.[†]

21.3 Products and coproducts of categories

Our next universal properties are products and coproducts.

Things To Think About

T 21.7 What do you think products and coproducts in **Cat** are? Can you prove it? Is this coherent with products of posets and monoids expressed as categories?

Given (small) categories $\mathbb{C}$ and $\mathbb{D}$ we define their cartesian product $\mathbb{C} \times \mathbb{D}$ based on the cartesian products of their sets of objects and sets of morphisms.

So the underlying graph of $\mathbb{C} \times \mathbb{D}$ will be this:

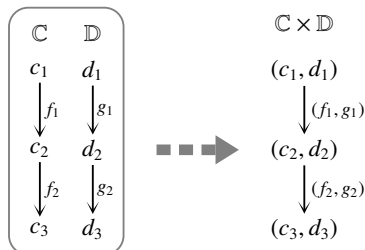
$$C_1 \times D_1 \xrightarrow[s \times t]{f \times g} C_0 \times D_0$$

That is, objects are ordered pairs (c, d) where $c \in \mathbb{C}$ and $d \in \mathbb{D}$, and a morphism $(c_1, d_1) \rightarrow (c_2, d_2)$ is an ordered pair (f, g) of morphisms as shown on the right.

$$\begin{array}{lcl} c_1 & \xrightarrow{f} & c_2 & \in \mathbb{C} \\ d_1 & \xrightarrow{g} & d_2 & \in \mathbb{D} \end{array}$$

Composition is “componentwise”. This means that to find the composite in $\mathbb{C} \times \mathbb{D}$ we take the composite in $\mathbb{C}$ and the composite in $\mathbb{D}$ individually, and then put them into an ordered pair.

I think it’s helpful to have the following sort of picture in mind, where the dashed arrow indicates how the individual arrows in $\mathbb{C}$ and $\mathbb{D}$ combine to make arrows in $\mathbb{C} \times \mathbb{D}$. Personally I find this diagram more compelling than the formula.



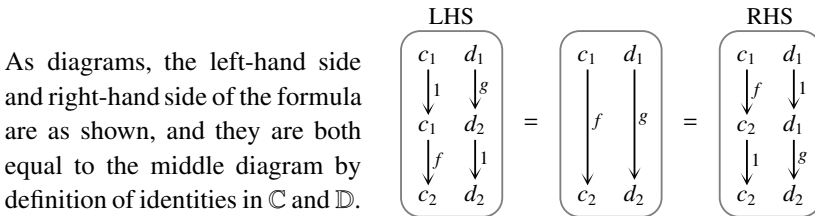
[†] In case you want to look it up: this all follows from general theorems about functors that are part of an adjunction, such as free and forgetful functors.

But anyway here's the formula: $(f_2, g_2) \circ (f_1, g_1) = (f_2 \circ f_1, g_2 \circ g_1)$.

Identities are also “componentwise”, which means $1_{(c,d)} = (1_c, 1_d)$.

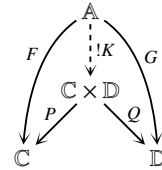
Things To Think About

T 21.8 The following formula is a useful result about product categories. Try turning it into a diagram in the spirit of the one above, making it rather more obvious why it's true: $(f, 1) \circ (1, g) = (1, g) \circ (f, 1)$.



This may look innocuous but it's actually quite profound and is related to many deep structures including what is called “interchange” laws at higher dimensions.

We still need to prove that $\mathbb{C} \times \mathbb{D}$ is a categorical product, by exhibiting its universal property. As usual we consider a diagram as shown on the right, and find a unique factorization K . Here P and Q are projection functors.



The substance of the proof is really no different from the proof in **Set**, it's just that we have to do it at the level of morphisms as well as objects, and make sure that functoriality holds. I'll just sketch it now.

In order to be a factorization the putative functor K must be defined as follows:

- on objects: $Ka = (Fa, Ga)$
- on morphisms: $Kf = (Ff, Gf)$ (which does type-check[†]).

Functoriality follows from functoriality of F and G individually, and the factorization is unique by construction. This proof is abstractly very similar to the proofs we saw for products of posets (expressed as categories with at most one morphism between any two objects) and of monoids (expressed as categories

[†] Remember: this means checking that the symbols do at least live in the right “homes”. In this case if we start with $a \xrightarrow{f} b$ then Kf is supposed to be a morphism $Ka \longrightarrow Kb$ in $\mathbb{C} \times \mathbb{D}$, and that's what we can type-check.

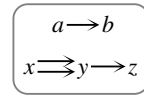
with only one object). This similarity is a sign that there is some further abstract explanation of things, but it's a little beyond the scope of this book.[†] But more specifically we can observe:

- Products of posets in the category **Pst** correspond to products of posets expressed as categories, in **Cat**.
- Products of monoids in the category **Mnd** correspond to products of monoids expressed as categories, in **Cat**.

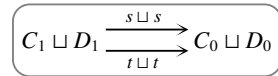
This can be expressed abstractly by saying that the functors **Pst** $\rightarrow$ **Cat** and **Mnd** $\rightarrow$ **Cat** preserve products.

For coproducts of categories we do something very similar: take the disjoint union of the objects and of the morphisms.

In pictures we get things like the category shown on the right, with two separate disconnected components. This example is a coproduct of the top part and the bottom part; it looks like two separate categories, which is sort of the point.



The underlying graph of a coproduct $\mathbb{C} \sqcup \mathbb{D}$ will then be as shown on the right.



(The notation $s \sqcup s$ means that the C_1 part just maps to the C_0 part by s as before, and the D_1 part just maps to the D_0 part by s as well. Likewise for $t \sqcup t$.) So we have the disjoint union of objects and the disjoint union of morphisms, with no morphisms between objects of $\mathbb{C}$ and objects of $\mathbb{D}$. Composition and identities are then those of $\mathbb{C}$ and $\mathbb{D}$; there is no interaction between the $\mathbb{C}$ part and the $\mathbb{D}$ part so there's nothing subtle about composition.

Coproducts of categories are thus much easier than coproducts of monoids, even though monoids are one-object categories. If you're wondering why coproducts of categories are easier, that's a very good question.

Things To Think About

T 21.9 See if you can understand why coproducts of monoids are not the same as coproducts of one-object categories. There are two flavors of answer to this question: the idea and the formalism.

The basic idea here is that the coproduct of two one-object categories has two objects, not one object, and this means that no arrows become composable

[†] In case you want to look it up, it's to do with these structures all being algebras for monads. This book will cover all the technical background needed to understand that, but will not actually cover that.

that weren't already composable. Whereas in the coproduct of two monoids all the elements can be combined using the binary product; essentially it's like taking the coproduct of one-object categories and then forcing the two single objects to be the same, making everything composable.

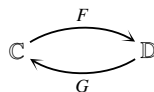
21.4 Isomorphisms of categories

Now that we have a category of (small) categories and functors we can try thinking about isomorphisms inside that category. By definition, an isomorphism in **Cat** must be a functor with an inverse functor, but the question is what that *means*: how we might characterize that directly, and what relationship it gives us between isomorphic categories.

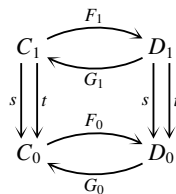
Things To Think About

T 21.10 Can you work out a way to characterize an invertible morphism in **Cat**? Think about the fact that functors are a generalization of functions, operating at the level of objects and morphisms, and that invertible functions are those that are both injective and surjective. It might help to think about the underlying graphs.

Let us think about a pair of functors as shown on the right, where F and G are inverses.



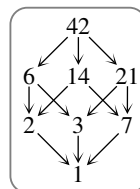
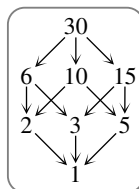
If we think what this does on the underlying graphs of $\mathbb{C}$ and $\mathbb{D}$ we get the diagram of sets and functions shown here. Note that I have drawn the underlying graphs vertically to emphasize the relationship with the diagram above.



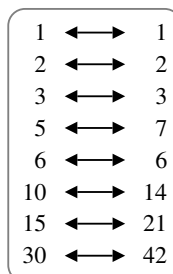
Now, for F and G to be inverses they must compose to the identity functor both ways round. We know that the identity functor acts as the identity *function* on both objects and morphisms, and that composition of functors happens by composition of the functions at the level of objects and morphisms. So F_1 and G_1 must compose to the identity function both ways round, and similarly F_0 and G_0 . This says we must have a bijection at the level of objects and a bijection at the level of morphisms, as well as the usual functoriality.

In practice this means that the categories $\mathbb{C}$ and $\mathbb{D}$ have the exact same structure, just with different names on the objects and morphisms. This is just like when we said an isomorphism of groups shows that two groups have the same pattern in their multiplication table, just with different names for elements.

For example the categories of factors shown on the right are isomorphic.



The bijection works on objects as shown on the right. In this case once we've determined the action on objects the action on morphisms follows: there is at most one morphism between any two objects, so no choice about where it goes.

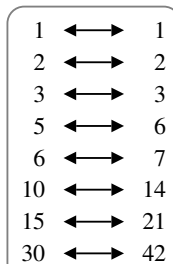


Things To Think About

T 21.11 In fact these categories are not *uniquely* isomorphic.

1. What other isomorphisms are there? That is, how else could we match up the objects?
2. Why can't we use the bijection on objects shown on the right, where the numbers go in size order on both sides?

Note that there are two types of answer to both of these questions: the technical one, where we simply give the formal details of the answer, and the "moral" one, where we give the deep explanation of the answer.



One way of thinking about this is to think about the symmetry in the actual cube diagrams. We know that the top and bottom vertices play special roles, but if we keep those cubes standing on one corner we can spin them, and each set of three vertices that are on the same level will exchange positions. This symmetry is a manifestation of the fact that the three prime factors play analogous roles here, so as long as we map the primes to the primes, and then the products to the analogous products, the structure will be preserved.

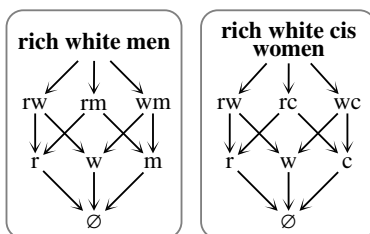
A more abstract approach is to observe that this cube is a product of three copies of the "quintessential arrow" category, one for each distinct prime. If we call that category I (for interval) then we are looking at $I \times I \times I$. From the symmetry of products we can deduce that there is an isomorphism permuting the copies of I in any way we like, which gives an isomorphism between these categories sending primes to primes.

This shows that saying that two categories “are isomorphic” is different from actually specifying an isomorphism between them. In the first case we are just saying that it is possible to match the structures up, and in the second case we are specifying a particular way of matching them up. That difference, as a general idea, becomes more and more important in higher dimensions.

This more or less answers the second part of the thought: in the bijection matching everything up in size order, the prime 5 on the left is mapped to a non-prime on the right (and the prime 7 on the right is mapped to a non-prime on the left) which is “morally” why this won’t work as an isomorphism of categories: it doesn’t map primes to primes.

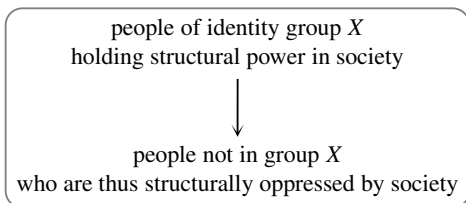
The technical reason it won’t work is that we will run into trouble when we try to make the functor act on morphisms: some of the morphisms won’t be able to go anywhere. If we define $F(2) = 2$ and $F(6) = 7$ then for example we will have a problem with the morphism $2 \rightarrow 6$. If F were a functor, this morphism would need to be mapped to a morphism $2 \rightarrow 7$ on the right, but there isn’t one. So this can’t be a functor, let alone an isomorphism of categories.

Analogously we have an isomorphism between the category of privilege involving “rich, white, male” and the category of women according to the three types of privilege “rich, white, cisgender”; these categories are depicted on the right (with abbreviations).



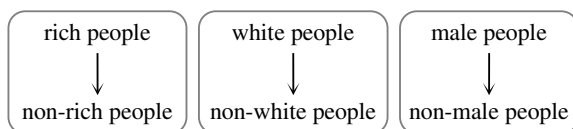
As with the factors of 30 and 42, different versions of this isomorphism are possible with different possible bijections on objects. This tells us which objects play analogous roles and which do not in these contexts. In particular we see that rich white cis women play an analogous role among women to the role that rich white men play in broader society.

As in the situation with factors, these cuboid categories are products of three instances of the single-arrow category shown on the right.

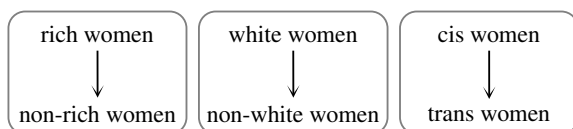


Thus the isomorphism between the cube categories comes down to the fact that we are starting from much simpler isomorphic categories.

In the first case we have these three “single arrow” categories.



In the second case we have these three categories, which are isomorphic to the previous ones.

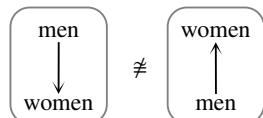


As a statement of category theory, the existence of the above isomorphisms of categories is not at all profound or subtle. The difficulty and subtlety about privilege is in how it manifests itself in life, not how the abstract structures are constructed. I just think that understanding abstract structures can help us focus on the subtlety in the right place rather than in an irrelevant place.

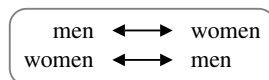
One subtlety is that arrows give us a much stronger correspondence between structures than if we’re just thinking about sets. I think that when we see some arrow diagrams as the same shape, our eyes are making a bijection visually, and they’re making a bijection on objects that respects the arrow structure.

Things To Think About

T 21.12 I like to point out that there is no isomorphism between categories pointing in the directions shown on the right. This is not a rigorous statement. Can you interpret it rigorously and prove it?



The rigorous statement is that the bijection on objects shown on the right cannot be extended to an isomorphism of categories.



The reason is that, as with when we tried to map 6 to 7 in the lattices of factors, there will be nowhere for the morphism between men and women to go.

This is an abstract explanation of the disagreement that happens when some people complain about the prevalence of men sexually harassing women, and some other people retort “Women do it to men too”. Women do do it to men too, but there is a structural difference when it’s a group that holds structural power in society harassing a group that does not hold structural power in society, as opposed to the other way round. Those who refuse to acknowledge this difference are looking at an isomorphism of sets as in the bijection above, and those who acknowledge the difference are looking at the impossibility of the isomorphism of categories based on that bijection, even if most people don’t express it in quite that way.

We can argue about *how* the structural power difference manifests itself in the experiences of men and women being harassed, and we can argue about whether men really do hold structural power in society (for which there is so much evidence that you have to be quite unreasonable to deny it, or perhaps not understanding what “structural power” means), but we really can’t validly argue about isomorphisms between such simple categories, at the abstract level.

One thing that we can and should call into question is whether this is the “right” notion of sameness for categories. We have seen that isomorphism is the “right” notion of sameness for objects *inside* a category, and that we therefore try not to invoke equalities between objects inside a category, rather than isomorphisms. However, now we are thinking about sameness for categories themselves. It turns out that in the definition of an isomorphism of categories we have invoked equalities between objects inside those categories, showing that the definition is “too strict”.

Things To Think About

T 21.13 Where in the definition of invertible functor are we invoking an equality between objects?

In order to say that a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ is invertible it needs to have an inverse G , which means $GF = 1_{\mathcal{C}}$ and $FG = 1_{\mathcal{D}}$. Now we unravel what this means: for two functors to be equal they have to have the same action on objects and on morphisms.

So for the first equation, on objects we’re saying $\boxed{\forall c \in \mathcal{C}, GFc = c}$. We get something similar for the second equation. So we are invoking equalities on objects: uh-oh. This is a hint that the notion of sameness for categories “wants” to be a little weaker than this, where we only invoke isomorphisms between objects in the category and not equalities. It is acceptable to invoke equalities between morphisms in $\mathcal{C}$ and $\mathcal{D}$ because we don’t have a notion of isomorphism between morphisms.[†]

So the idea is that a better notion of sameness for categories would be via a functor with the following properties.[‡]

- On morphisms: strictly invertible, so $\boxed{\forall f \in \mathcal{C}, GFf = f}$ and similarly for the composite FG .
- On objects: only invertible up to isomorphism, so $\boxed{\forall c \in \mathcal{C}, GFc \cong c}$ and similarly for the composite FG .

We will think about the part on morphisms first, as that is more straightforward. It brings us to the concept of full and faithful functors.

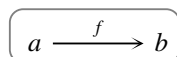
[†] This requires higher-dimensional categories, as we’ll see.

[‡] Notation reminders: $\forall$ means “for all”, $\in$ means “in”, $\cong$ means “is isomorphic to”.

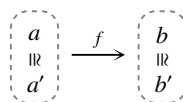
21.5 Full and faithful functors

If we are dealing with a functor that is not strictly a bijection on objects we need to be careful what “bijection on morphisms” means. We’re going to warm up by thinking about the “quintessential arrow” category, but adding in some isomorphic copies of the two objects. Categorical instinct says that this new category isn’t really different. First we need to make these thoughts precise.

We consider a category with two objects and one morphism between them, as shown on the right.



Now we’ll try to make a version of this category which is essentially the same, but has two isomorphic versions of a , and two isomorphic versions of b . (The symbol $\cong$ is an isomorphism sign $\cong$ on its side.)

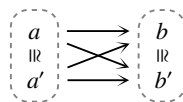


Things To Think About

T 21.14 Can you turn this vague idea into something rigorous, by working out what all the arrows should be to make sense of this idea? We need various arrows from a and a' to b and b' .

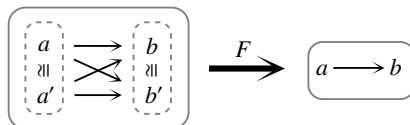
The idea is that the morphism f has one manifestation from each isomorphic copy of a to each isomorphic copy of b , but they don’t really count as “different”, just as the isomorphic objects don’t really count as “different”.

If we continue to represent the isomorphisms by the squiggles then the rest of the arrows are four versions of f as shown on the right (not drawing identities).



This category should count as “the same” as the first one, as we’ve just included some more isomorphic objects. It just sort of fattens up the category a bit. We’ll now try to make that precise. We are going to investigate the following functor which “collapses” the fattened category back to the original lean one.

Isomorphic objects are mapped to the same place, and all the versions of f are mapped to the same place.



Things To Think About

T 21.15 Can you make precise a sense in which F is a “bijection up to isomorphism” on objects? Also can you see that it is not a bijection on morphisms, but somehow gives a perfect correspondence on morphisms in the context?

We could say F is a “bijection up to isomorphism” on objects, in the sense that objects are only mapped to the same place if they were already isomorphic.

However it is not a bijection on morphisms as four morphisms are sent to the same place, so it is not injective on morphisms.

However, it *is* a bijection on morphisms if we look at individual homsets.[†] By definition, a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ gives functions on homsets as follows: given any pair of objects $x, y \in \mathcal{C}$, we have a function $\mathcal{C}(x, y) \rightarrow \mathcal{D}(Fx, Fy)$. It is these functions that are all bijections in our example, because once we restrict our attention to a particular pair of objects on the left, we don't see the "extra" copies of the morphism f any more. If you're not sure about this it's worth trying it out for different pairs of objects on the left until you see the point. The overall idea is that this functor is really encapsulating the sense in which these two categories are not *really* different.

The important conclusion here is that we should not look at the set of morphisms overall, but we should look at individual homsets. This comes down to the difference between the two definitions of category we've seen:

- By homsets: for every pair of objects a, b a set of morphisms $\mathcal{C}(a, b)$.
- By total set of morphisms: a set C_1 of morphisms equipped with source and target functions to the set C_0 of objects.

It turns out that the first definition guides us better for thinking about the correct notion of "bijection on morphisms". For a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ we need to think about each function $\mathcal{C}(a, b) \rightarrow \mathcal{D}(Fa, Fb)$ individually, rather than thinking about the "total" function $C_1 \rightarrow D_1$.

Now that we know to think about bijections on homsets, we can unravel this a little further, and break down the concept of bijection into injection and surjection. It turns out that thinking about injectivity and surjectivity separately is fruitful here, so we give those concepts some names.

The following definition piles up notation and terminology. We'll try to remain calm and unravel it afterwards.

Definition 21.3 Let $F: \mathcal{C} \rightarrow \mathcal{D}$ be a functor.

- F is called *faithful* if for all $x, y \in \mathcal{C}$ the function $\mathcal{C}(a, b) \rightarrow \mathcal{D}(Fa, Fb)$ is injective. We also sometimes call this *locally injective*.
- F is called *full* if for all $x, y \in \mathcal{C}$ the function $\mathcal{C}(a, b) \rightarrow \mathcal{D}(Fa, Fb)$ is surjective. We also sometimes call this *locally surjective*.

Note that "locally" in abstract math typically means that we're zooming in to look at a situation close-up, and only paying attention to one small region at a time. In category theory it usually means we're doing something on homsets.

[†] That is, set of morphisms between a fixed pair of objects.

If for all x, y the function $\mathcal{C}(a, b) \rightarrow \mathcal{D}(Fa, Fb)$ is a bijection then we say F is full and faithful.[†]

Things To Think About

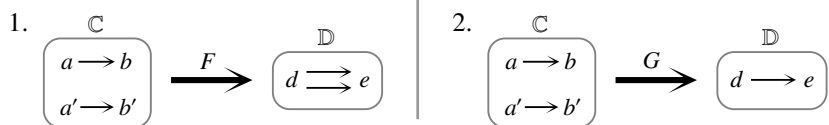
T 21.16 Can you unravel the definitions of injective and surjective in the above context to gain a closer understanding of what full and faithful mean?

In practice we often unravel the definitions as follows.

- Full: $\forall Fx \xrightarrow{h} Fy \in \mathcal{D}, \exists x \xrightarrow{f} y \in \mathcal{C}$ such that $Ff = h$.
- Faithful: $\forall f, g: x \rightarrow y \in \mathcal{C}, Ff = Fg \implies f = g$.
- Full and faithful: $\forall Fx \xrightarrow{h} Fy \in \mathcal{D}, \exists! x \xrightarrow{f} y \in \mathcal{C}$ such that $Ff = h$.

Things To Think About

T 21.17 Are the following functors full and faithful? (The diagrams below are not rigorous definitions, but I invite you to make the “obvious” definitions.)



My intention for the first functor is that the two morphisms on the left are sent to the two distinct morphisms on the right. So the functor is bijective on morphisms but not locally: it fails to be locally surjective. Intuitively, the homset $\mathcal{D}(d, e)$ on the right has two morphisms but the homsets on the left only have one morphism each, so local surjectivity is doomed to fail. Formally we could say $Fa = d$ and $Fb = e$ so the function $\mathcal{C}(a, b) \rightarrow \mathcal{D}(Fa, Fb) = \mathcal{D}(d, e)$ is not surjective.

The second functor is not full and faithful either but the situation is more subtle. This time all the obvious homsets have one morphism each so we need to look more closely. The problem is that some completely unrelated objects on the left end up connected on the right. For example we can consider the function $\mathcal{C}(a, b') \rightarrow \mathcal{D}(Fa, Fb') = \mathcal{D}(d, e)$. The homset on the left is empty but the one on the right has one element, so this function can't be surjective.

In fact a clue to this is that the functor fails to be “injective on objects”, even up to isomorphism: $Fa = Fa'$ but a and a' are not even isomorphic (so

[†] Some people call it “fully faithful” but others of us find it odd to make it sound as if the full part is modifying the faithful part. Those really are two separate properties. It would be like calling a bijective function “surjectively injective”. Or, as John Baez put it “If I cheat on my wife that’s a separate issue from the fact that I ate a big dinner”. I can’t find a citation for this so I suspect I heard him say it in person.

we could have used the homset $\mathbb{C}(a, a')$ to show that the functor is not locally surjective). We will come back to this idea.

Recall in Section 20.5 we mentioned the idea of reflecting structure, and showed that all functors preserve isomorphisms but functors do not in general reflect isomorphisms.

Things To Think About

T 21.18 Show that a full and faithful functor reflects isomorphisms. As usual the first step is to write down rigorously what this means.

Proposition 21.4 *Let $F: \mathbb{C} \rightarrow \mathbb{D}$ be a full and faithful functor and $a \xrightarrow{f} b$ be a morphism in $\mathbb{C}$. Then f is an isomorphism iff Ff is an isomorphism.*

Note that this is “preserves and reflects” in one statement; the “reflects” part is the “if” part, that is: if Ff is an isomorphism then f is an isomorphism. So we’ll just prove that part here, as we’ve done “preserves” before.

Proof Suppose $Fa \xrightarrow{Ff} Fb$ is an isomorphism in $\mathbb{D}$, with inverse $Fb \xrightarrow{h} Fa$. We aim to find an inverse for f in $\mathbb{C}$.

The idea is that morphisms $Fb \rightarrow Fa$ in $\mathbb{D}$ precisely correspond to morphisms $b \rightarrow a$ in $\mathbb{C}$ since F is full and faithful, and so h must have come from an inverse in $\mathbb{C}$ under that correspondence.

We know F is full and faithful, so there is a unique morphism $b \xrightarrow{g} a \in \mathbb{C}$ such that $Fg = h$. We claim that g is an inverse for f .

We need to compose them and show that the result is the identity in $\mathbb{C}$. If we apply F to the composite we get the identity in $\mathbb{D}$; then we show that F being faithful means that the only thing that can map to the identity is the identity.

First consider the composite $g \circ f$. We know that

$$\begin{aligned} F(g \circ f) &= Fg \circ Ff && \text{by functoriality} \\ &= h \circ Ff && \text{since we defined } g \text{ by } Fg = h \\ &= 1_{Fa} && \text{since we defined } h \text{ as the inverse of } Ff \\ &= F1_a && \text{by functoriality.} \end{aligned}$$

Since F is faithful, we must have[†] $g \circ f = 1_a$ and similarly we can show $f \circ g = 1_b$. So g is an inverse for f and F reflects isomorphisms as claimed. $\square$

We will come back to this when we think about “essentially injective”, and also when we think about the Yoneda embedding.

[†] This follows from the definition of injectivity.

Things To Think About

T 21.19 Earlier on we said that if we express posets as categories then the functors between them are precisely the order-preserving functions. Likewise if we express monoids as one-object categories then the functors between them are precisely the monoid homomorphisms. What is this saying about the “inclusion” functors $\mathbf{Pst} \xrightarrow{K} \mathbf{Cat}$ and $\mathbf{Mnd} \xrightarrow{\Sigma} \mathbf{Cat}$?

The formal version of these statements is that the functors K and Σ are full and faithful. Consider posets A and B . We are saying that the functors $KA \rightarrow KB$ are *precisely* the order-preserving functions $A \rightarrow B$, which says that this function on homsets is a bijection $\mathbf{Pst}(A, B) \rightarrow \mathbf{Cat}(KA, KB)$, that is, K is full and faithful. Likewise for Σ and monoids.

The notion of “full and faithful” has given us a good notion of a functor being an “isomorphism at the level of morphisms”. For the level of objects we mentioned above the idea of a “bijection up to isomorphism”. The idea is to modify the definition of a bijection. Recall that a function $f: A \rightarrow B$ is a bijection if it is injective and surjective, that is:

1. Injective: $f(a) = f(a') \implies a = a'$.
2. Surjective: $\forall b \in B, \exists a \in A$ such that $f(a) = b$.

Now if we are in a category these definitions are too strict for the level of objects as they have equalities all over them. We make the following definition instead. You might wonder why we’ve mysteriously ignored the “injectivity” part; we’ll come back to that question shortly.

Definition 21.5 We say that a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ is *essentially surjective* (on objects) if $(\forall d \in \mathcal{D}, \exists c \in \mathcal{C} \text{ such that } f(c) \cong d)$.

This is an example of a key principle in “categorification”, where we take an axiom and replace all the equalities between objects by isomorphisms. We then have this definition of “bijective up to isomorphism” for a functor:

Definition 21.6 A functor $F: \mathcal{C} \rightarrow \mathcal{D}$ is a *pointwise equivalence* if it is full and faithful, and essentially surjective on objects.

Things To Think About

T 21.20 Why don’t we need something like “essentially injective on objects”? That is, can you write down what this might mean and show that it is satisfied by any full and faithful functor.

We can make up a definition of “essentially injective on objects” by replacing the equalities by isomorphisms in the definition of injective. This would give something like this: $(\forall c, c' \in \mathcal{C}, Fc \cong Fc' \implies c \cong c')$.

But this is just a slightly weak version of “reflects isomorphisms”. It is weaker because it just talks about objects being *isomorphic* rather than about specific isomorphisms. In any case it certainly follows from the fact that full and faithful functors reflect isomorphisms.

Proposition 21.7 *Suppose $F: \mathcal{C} \rightarrow \mathcal{D}$ is full and faithful. Then F is “essentially injective on objects” as in the above definition.*

Proof Suppose we have an isomorphism $Fc \xrightarrow{h} Fc'$ in $\mathcal{D}$. Since F is full we know that $h = Ff$ for some $c \xrightarrow{f} c'$ in $\mathcal{C}$. But full and faithful functors reflect isomorphisms so f must be an isomorphism, so $c \cong c'$ as required. $\square$

Note that this result is not very good, because of its vagueness over “isomorphic” objects. The result about F reflecting isomorphisms is the morally correct statement. But in any case we see that “essentially injective on objects” actually follows from “locally bijective on morphisms”. This is a very interesting phenomenon meaning that in higher dimensions we don’t need to invoke injectivity at every dimension, only (a suitably weak notion of) surjectivity, as injectivity will always follow from the dimension above if there is one.

It is crucial to note that the definition of pointwise equivalence is a weakened version of bijection, not a higher-dimensional version of isomorphism. That is, it uses elements all over the place and so is not categorical, in the sense that we could not immediately place it in other categories. For a categorical version we need some higher-dimensional structure in our category of categories. We have taken equations between objects and weakened them, but what we really need to do to weaken the notion of invertible functor is take the equations in the definition of isomorphism and weaken those.

This would give a pair of functors as shown on the right, satisfying something like $GF \cong 1_{\mathcal{C}}$ and $FG \cong 1_{\mathcal{D}}$.

$$\mathcal{C} \begin{array}{c} \xrightarrow{F} \\ \xleftarrow{G} \end{array} \mathcal{D}$$

However, for this we need a notion of (iso)morphism between functors, which is one dimension up from what we have now, in a sense that we are about to investigate. If we could do it, it would give us a *categorical* definition of “weak isomorphism” for functors, that is, one that we can then take into any suitably higher-dimensional category. (We just need one more dimension, so that would be a 2-category.)

There are many situations in math where the full higher-dimensional situation is impractical, unwieldy or just unpalatable, so we try and make do with lower-dimensional structures that somehow encapsulate some features of the higher-dimensional situation. That is what pointwise equivalence does, and it is very useful, but we will look more at the true higher-dimensional structures next.